

INF01151 – Sistemas Operacionais II

Aula 25 - Replicação

Prof. Alberto Egon Schaeffer Filho

Introdução

- Replicação é a manutenção de várias cópias (réplicas) dos dados ou serviços em vários computadores distribuídos
 - Implica réplica de servidores utilizadas para “melhorar” um serviço
- Vantagens da replicação
 - Melhor desempenho
 - Aumento da disponibilidade
 - Tolerância a falhas
- Como replicação é feita?
 - Replicação passiva, replicação ativa, ...

① Replicação para desempenho

- Princípio de funcionamento
 - Cópias extras podem permitir dividir workload entre múltiplos servidores
 - Cópias extras permitem que dados fiquem disponíveis mais perto dos clientes
- Replicação de dados
 - Imutáveis
 - Trivial, pois não há necessidade de atualizações
 - Aumenta o desempenho com custo relativamente baixo
 - Dinâmicos
 - Necessidade de mecanismos (protocolos) para garantir que os clientes recebam dados atualizados
- Tipicamente é boa quando há mais *reads*
 - *Read* pode ser feito a partir da melhor cópia
 - *Writes* devem ir para todas as cópias

② Replicação para disponibilidade

- Usuários necessitam ter sistemas com alta disponibilidade
 - Proporção de tempo no qual um serviço (dados) é disponível a usuários dentro de um tempo razoável (ideal é 100%!!)
- Além do tempo gasto no controle de concorrência (ex., devido a locks), outros aspectos afetam a disponibilidade
 - Particionamento da rede e desconexões
 - Falha no servidor
- Múltiplas cópias podem garantir que falhas não impossibilitem utilização do sistema
 - Possível aumentar a disponibilidade através de replicação
 - Ex.: n servidores, probabilidade de falha no servidor igual a p , portanto, a disponibilidade do serviço é dada por $1-p^n$
 - com $p=5\%$, então 1 servidor fornece 95%, 2 servidores 99,75%, etc

③ Replicação para tolerância a falhas

- Alta disponibilidade é interessante, mas não implica em dados necessariamente corretos
 - Desatualização
 - Atualizações conflitantes em caso de particionamento da rede
 - Usuários em lados opostos de uma rede particionada fazem atualizações conflitantes que precisam ser resolvidas
 - Servidores podem fazer coisas erradas, proteção contra falhas bizantinas
- Serviço tolerante a falhas
 - Garante um comportamento correto mesmo na presença de um certo tipo e número de falhas
 - Se f em $f+1$ servidores falham (colapso), então em princípio, pelo menos uma cópia permanece para atender o serviço
 - Se f falham de forma bizantina, $2f+1$ servidores oferecem serviço correto, se os servidores corretos concordarem na resposta (votação)

Modelo de replicação: requisitos

- Requisitos comuns de um modelo de replicação de dados
 - **Transparência**: clientes não vêem que existem múltiplas cópias físicas
 - Mesmo que existam múltiplas cópias físicas, cliente só conhece um único objeto lógico
 - Clientes apenas requisitam operações a esse objeto lógico
 - Clientes esperam por apenas um resultado (mesmo que operação tenha executado em mais de uma cópia)
 - **Consistência**: múltiplas cópias devem estar coerentes
 - Dependente da aplicação (pode ser mais restrito ou menos restrito)
 - Relação custo x benefício
 - Inconsistência temporária é uma possibilidade
- Objetivo do sistema distribuído
 - Distribuir updates para todas as réplicas
 - Direcionar requisições para a réplica mais “conveniente”

Modelo de replicação

- Modelo básico:
 - Dados são conjuntos de itens chamados de objetos (arquivo, objeto Java, etc)
 - Cada objeto lógico é formado por um conjunto de cópias físicas (réplicas)
 - Réplicas não são necessariamente idênticas/consistentes
 - Algumas podem ter sido atualizadas enquanto outras não
 - Gerenciadores (replica managers) aplicam operações em réplicas
 - De forma recuperável
 - De forma atômica
 - Resultado depende
 - Do estado da réplica
 - Do ordenamento das operações
- Replicação pode ser utilizada em combinação com **transações**

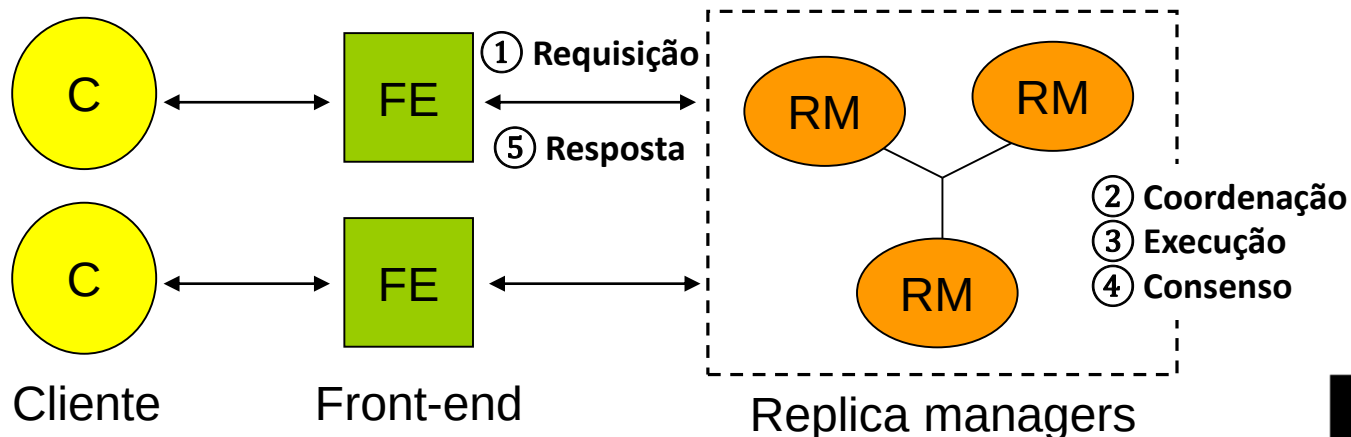
Modelo (genérico) de replicação

- Arquitetura básica:

Podem ser satisfeitas por qualquer réplica

Precisam ser propagadas para todas as réplicas

- **Clientes (C):** fazem requisições (*read-only* ou *update requests*)
- **Front-end (FE):** fornece transparência aos clientes encaminhando requisições aos RMs
- **Replica managers (RMs):** executam as operações nas réplicas de forma atômica
 - Se aplicado em ambiente cliente-servidor, cada RM é um servidor
 - Considera um número estático ou dinâmico de RMs



Cinco etapas (genérico)

1. **Requisição**: envio da requisição do FE a um ou mais RMs
 - Para um RM primário, que se comunicará com os demais; ou
 - Em *multicast* para os RMs
2. **Coordenação**: RMs definem se a requisição deve ser processada, e a ordem de execução de requisições entre si
 - Ordem FIFO: baseado na ordem dos requests em um dado FE
 - Ordem causal: baseado na ordenação happened-before
 - Ordem total: baseado na mesma ordem em todos os RMs
3. **Execução**: realiza requisição de forma *tentativa*, de modo que essa possa ser desfeita mais tarde se necessário (*transação*)
4. **Consenso**: RMs efetivam (*commit*) ou abortam (*abort*) a execução
5. **Resposta**: um ou mais RMs respondem ao FE
 - Primeira que chega
 - Resposta da maioria (falha bizantina)

Noções de comunicação em grupo

Implementação da replicação

- Serviço baseado em replicação: ***visão idealista***
 - Serviço continua respondendo apesar de eventuais falhas
 - Clientes não conseguem distinguir se serviço é obtido de implementação com dados replicados ou de uma única instância
- Considerando uma solução trivial (e ingênua)
 - Dois gerenciadores de réplica A e B que mantêm cópias de duas contas bancárias x e y
 - Clientes lêem e atualizam as contas em um gerenciador de réplica e tentam o outro em caso de problemas
 - Gerenciadores de réplica propagam as informações de atualização em background, após responder para os clientes
 - As contas inicialmente tem saldo zero

Situação exemplo

- Cliente 1
 - Atualiza o saldo da conta x para \$1 no gerenciador de réplica B (que falha logo em seguida)
 - Atualiza o saldo da conta y para \$2, mas gerenciador de réplica B não responde
 - Executa então no gerenciador de réplica A
- Cliente 2
 - Lê o saldo das contas x e y no gerenciador de réplica A

Cliente 1	Cliente 2
setBalance _B (x, 1)	
setBalance _A (y, 2)	
	getBalance _A (y) → 2
	getBalance _A (x) → 0

Comportamento anômalo não teria acontecido se as contas de banco tivessem sido implementadas em um servidor único

Linearizability vs. Sequential consistency

- Linearizability

- Sistema distribuído que atua exatamente como um único servidor gerenciando uma única cópia é *linearizable*
- Deve satisfazer dois critérios para qualquer execução
 1. Série de operações intercaladas atende a especificação de uma única cópia correta
 2. Intercalação é consistente com os tempos reais em que as operações ocorreram na execução
- Inclui noção de tempo real, o que é difícil de obter no mundo real

- Sequential consistency

- Garantir que as operações ocorrem de forma correta e em determinada ordem
 - Sem uso de relógios globais
- Deve satisfazer dois critérios para qualquer execução
 1. Série de operações intercaladas atende a especificação de uma única cópia correta
 2. Intercalação é consistente com a ordem do programa executado por cada cliente em específico (intercalações podem embaralhar sequencia de operações de um conjunto de clientes em qualquer ordem, desde que a ordem das operações de cada cliente seja mantida)
- Cada cliente vê as operações na ordem em que ele fez elas

Linearizability vs. Sequential consistency

Linearizable

Cliente 1	Cliente 2
	$\text{getBalance}_A(y) \rightarrow 0$
	$\text{getBalance}_A(x) \rightarrow 0$
$\text{setBalance}_B(x, 1)$	
$\text{setBalance}_A(y, 2)$	

Linearizable $\Rightarrow$ sequentially consistent

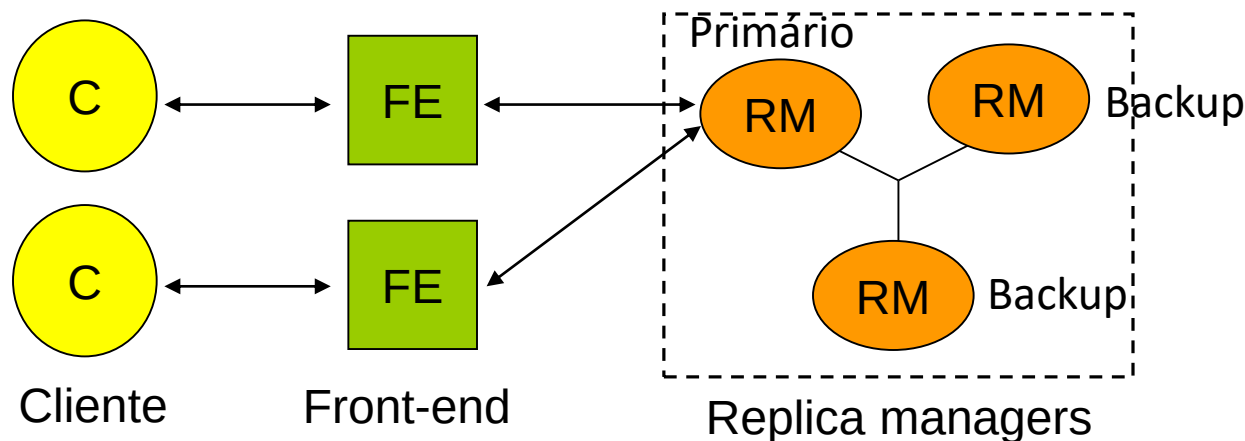
Sequentially consistent, mas não linearizable

Cliente 1	Cliente 2
$\text{setBalance}_B(x, 1)$	
	$\text{getBalance}_A(y) \rightarrow 0$
	$\text{getBalance}_A(x) \rightarrow 0$
$\text{setBalance}_A(y, 2)$	

- Valor de x consultado depois que ele foi alterado para 1!
- Sequentially consistent pois:
 - Estado que cliente 2 vê realmente existiu antes do cliente 1 iniciar sua operação
 - Cliente 2 "poderia" ter executado suas operações antes do cliente 1 ter iniciado

Replicação passiva

- Replicação passiva ou *backup*-primário
 - Um único gerenciador de replica (RM) primário
 - Um ou mais gerenciadores de réplicas secundários (backup ou escravos)
- FE comunica somente com o RM primário para executar serviço
 - RM primário executa as operações e envia atualizações aos secundários
 - Se o primário falha, um secundário assume (eleição)



Replicação passiva: etapas (1-5)

- **Requisição:**
 - FE envia a requisição, contendo um identificador único, ao RM primário
- **Coordenação:**
 - Primário trata requisições de forma atômica, na ordem em que as recebe
 - Verifica o identificador único, em caso de já ter executado, simplesmente reenvia a resposta
- **Execução:**
 - Realiza a requisição e armazena o resultado
- **Consenso:**
 - Se a requisição atualizar algum dado, o primário envia o estado atualizado, a resposta e o identificador único para todos os secundários
 - Espera confirmação dos secundários
- **Resposta:**
 - Primário envia resposta ao FE, que a encaminha ao cliente

Replicação passiva: linearizability

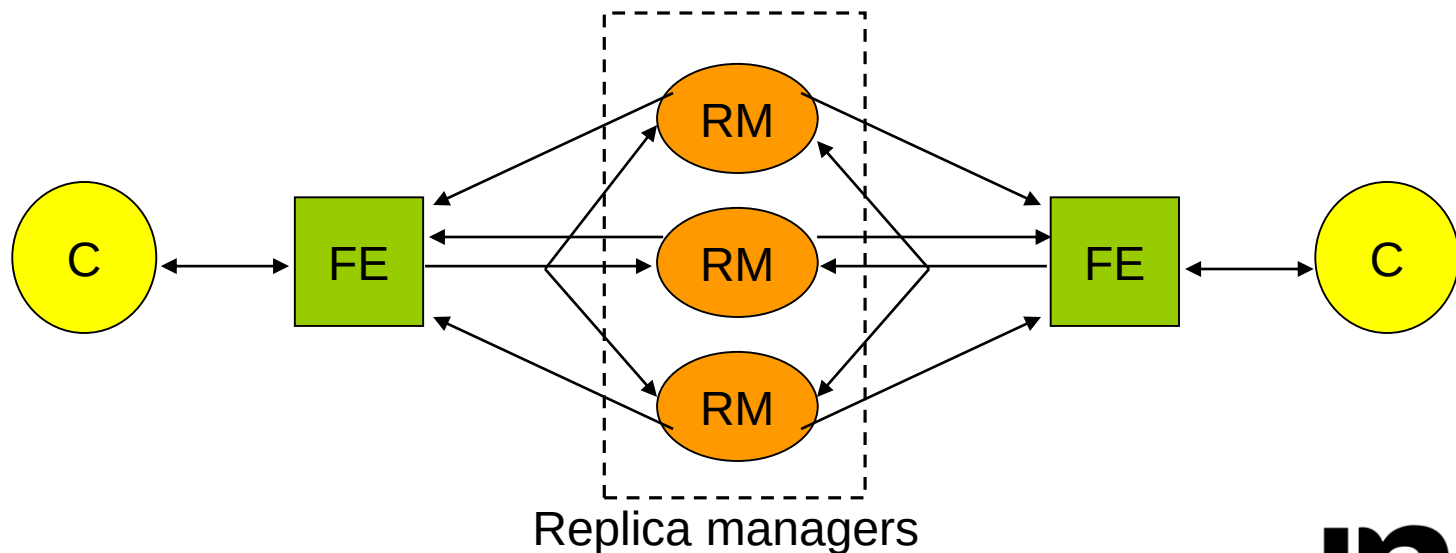
- Claramente satisfaz linearizability se não houver falhas
 - Primário realiza operações em “tempo real” e em ordem
 - Não há preocupação em atualizar em múltiplos lugares
- E se houver falhas?
 - Ainda satisfaz linearizability se:
 - Um único backup se tornar o novo primário
 - Os outros replica managers sobreviventes concordam em quais operações haviam sido realizadas antes da falha
- Possível utilizar características da comunicação em grupo para satisfazer essas características

Replicação passiva: comentários

- Aspectos de tolerância a falhas
 - Necessita $f+1$ réplicas para sobreviver f falhas por colapso
 - FE apenas precisa ser capaz de localizar novo primário
 - Não tolera falhas bizantinas!
 - Falha bizantina no primário causa caos em todo o sistema
 - Replicação passiva exige comportamento correto do primário

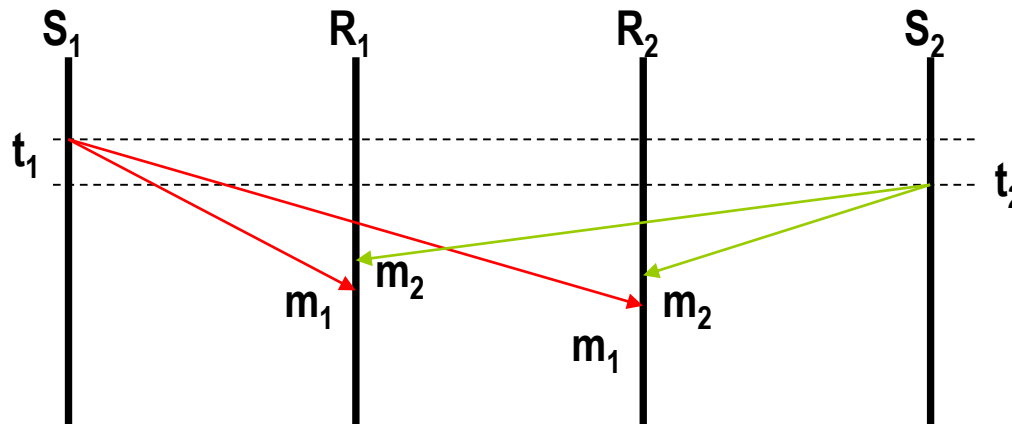
Replicação ativa

- FEs fazem *multicast* da requisição para o grupo de RMs
 - RMs executam as requisições de forma independente e respondem ao FE
 - RMs são todos equivalentes
 - Se qualquer RM sofrer colapso, não há impacto no desempenho, e RMs restantes irão responder da forma esperada
 - Garantia de ordem e funcionamento é o da semântica de *multicast*



Multicast totalmente ordenado

- Se um processo $p \in g$ executa $deliver(m)$ antes de $deliver(m')$, então qualquer outro processo $q \in g$ entregará m' após m
 - Garantia que todas as mensagens são entregues na mesma ordem em todos os processos mesmo que difira da ordem de emissão



Replicação ativa: etapas (1-3)

- **Requisição:**

- FE cria uma identificação única para as requisições e envia (*multicast*) para o grupo de gerenciadores de réplica
 - Multicast **totalmente ordenado** e **confiável**

- **Coordenação:**

- Semântica do multicast garante que todas as réplicas receberam a requisição na mesma ordem (total)

- **Execução:**

- Cada RM executa a requisição, e como todas são recebidas na mesma ordem, todos RMs ativos (e corretos) processam a requisição da mesma forma

Replicação ativa: etapas (4-5)

- **Consenso:**
 - Não há necessidade de consenso devido a semântica de multicast empregada
- **Resposta:**
 - Cada RM envia a resposta ao FE
 - Número de respostas coletadas pelo FE depende das condições de falhas previstas
 - Opções?

Replicação ativa: sequential consistency

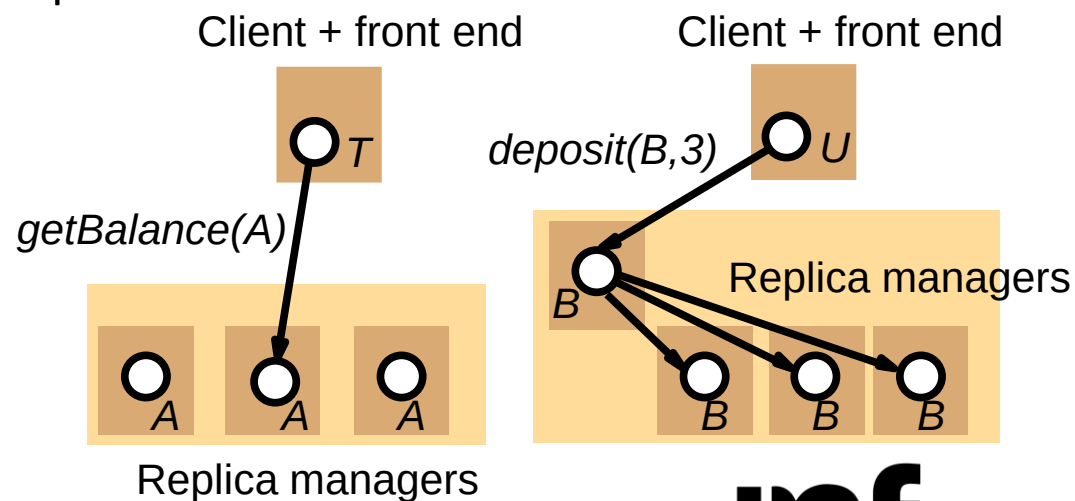
- Replicação ativa não suporta linearizability
 - Ordem total em que os RMs executam as requisições não é necessariamente consistente com a ordem em tempo real que clientes fizeram requisições
- Replicação ativa suporta sequential consistency
 - Requisições feitas em cada FE são processadas em ordem, uma por vez
 - Todas os RMs corretos processam a mesma sequência de requests

Atualização de réplicas: outras técnicas

- Protocolos disponíveis
 - Read-only
 - Read-one/write-all
 - Primary-copy
 - Quorum based

Read-only e read-one/write-all

- *Read-only*
 - Usado em conjunto de dados imutáveis
 - Lê de qualquer cópia e por definição não há escritas
- *Read-one/write-all*
 - Leitura é feita em qualquer réplica
 - Escrita é feita em todas as réplicas
 - Problema:
 - *Lock* em todas as réplicas para se fazer uma escrita
→ gargalo
 - Todas as réplicas devem estar disponíveis



Primary-copy

- Primary-copy
 - Uma réplica é destacada para ser a primária
 - Read é feito em qualquer cópia
 - Write é feito na primária
 - Servidores com cópias secundárias
 - Solicitam atualizações (*eager* vs. *lazy*)
 - São informados das alterações
 - **Problema:** semântica da consistência de dados

Quorum-based

- Problema comum às técnicas anteriores é particionamento da rede
 - Abordagem **pessimista**: previne que inconsistências ocorram durante particionamento, e quando restaurado, apenas atualiza cópias
- Uso de quorum: n réplicas de um objeto O
 - Leitura: para ler é necessário ler **no mínimo r réplicas** (*read quorum*)
 - Escrita: necessário fazer **no mínimo em w réplicas** (*write quorum*)
- Condição necessária: **$r + w > n$ & $w > n/2$**
 - Deve haver no mínimo uma réplica que esteja simultaneamente no quorum de leitura e de escrita
- Algumas réplicas podem ficar desatualizadas. Como determinar?
 - Necessário identificar a réplica mais atual
 - Número de versão em cada réplica que é atualizado a cada modificação
 - Réplica com o maior número de versão é a mais atual

Leitura e escrita baseada em *quorum*

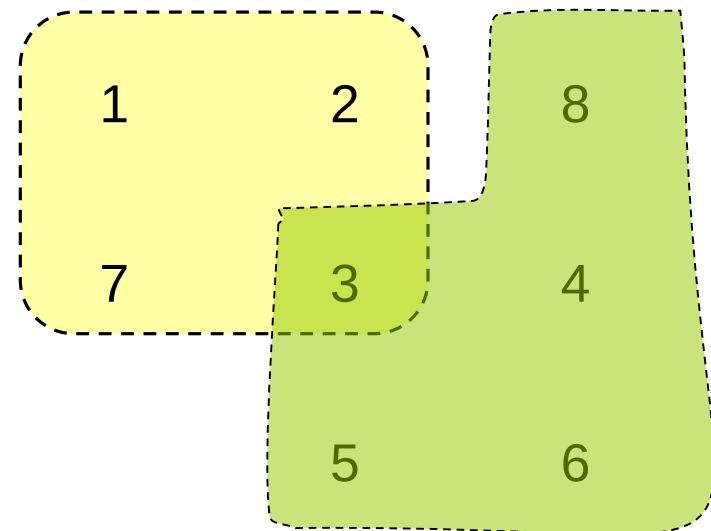
- **Read**

- Recupera as r réplicas de leitura
- Seleciona réplica com a maior versão
- Lê dado da réplica selecionada

- **Write**

- Recupera as w réplicas
- Seleciona réplica com a maior versão
- Incrementa número de versão
- Escreve novo valor e nova versão em todas as réplicas

$n = 8; r = 4; w = 5$



Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“*Distributed Systems: Concepts and Design*” (5th edition),
Addison Wesley, 2012
 - Capítulo 18